



ETHNOTECH ACADEMIC SOLUTIONS PVT LTD

Java Full Stack Development Program — Final Examination

EXAM SET 3

RESTAURANT ORDER MANAGEMENT

QUESTION 1: CORE JAVA FOUNDATIONS [10 Marks]

A restaurant billing counter needs a standalone module to compute a customer's order total, honoring a promotional 'bulk order' giveaway rule.

Requirements:

- Create a model class named `OrderItem` containing primitive/object fields: `int itemId`, `String itemName`, `int quantity`, `double unitPrice`. Provide a parameterized constructor to initialize these values.
- Create a utility class named `OrderBillEngine`. Within it, write a method with the exact signature shown below.

Method Signature:

```
public static double computeOrderTotal(OrderItem[] items, int freeItemThreshold) throws Exception
```

Implementation Logic:

The method must loop through the fixed-size array (`items`, allocated for 12 entries) using a standard primitive loop, skipping any null slots. For each valid item, compute `quantity * unitPrice` as its line value.

The Catch (Confusion Trap):

Any item whose quantity is greater than or equal to `freeItemThreshold` is a promotional giveaway — its line value must be added as 0 to the total (the item is still counted/processed, but contributes no cost). Only items below the threshold add their full `quantity * unitPrice` value. Return the final total as a raw primitive double.

QUESTION 2: JDBC INTEGRATION [8 Marks]

Write a Java data access block using native JDBC steps to connect to the restaurant database, perform a selective read operation, and populate an execution cache array while safely managing connections.

Requirements:

- Create a class named `OrderDataBridge` containing a data layer method named `fetchHighValueOrders`. The signature must be appended with `throws Exception` without any local try-catch handling blocks.
- Establish a connection to the database URL `jdbc:mysql://localhost:3306/restaurant_db` using a username of 'root' and a password of 'password'.
- Pipe a raw database statement query string: `"SELECT * FROM food_orders WHERE amount > 1000"`.
- Load the tracking outputs directly into a native array instance: `OrderItem[] highValueSet = new OrderItem[40]`.

Method Signature:

```
public static OrderItem[] fetchHighValueOrders() throws Exception
```

Implementation Logic:

Extract the database rows sequentially using `.next()` iterations, mapping columns into `OrderItem` instances, and storing them at consecutive positions in the `highValueSet` array without leaving gaps.

The Catch (Confusion Trap):

You must explicitly invoke `.close()` on your `ResultSet`, `Statement`, and `Connection` reference elements at the bottom of the method block, in the exact reverse order of their opening sequence, to prevent enterprise resources from hanging.

QUESTION 3: DATABASE INTEGRATION — MYSQL & MONGODB [7 Marks]

The restaurant wants both a relational sales ledger and a document-store aggregation to identify its best-selling menu items.

Part A — MySQL (Module 3) [4 Marks]**Table Creation:**

- order_id: Integer, primary key, auto-incrementing.
- table_number: Integer, cannot be null.
- item_name: Variable character string (up to 100 characters), cannot be null.
- amount: Decimal type (8 digits total, 2 decimal places), cannot be null, and must be configured with a check rule ensuring it is strictly greater than 0.

Data Ingestion:

Write an explicit INSERT statement adding a row with table_number 7, item_name 'Paneer Butter Masala', and amount 450.00.

Data Retrieval Query:

Write a query that returns item_name and the total amount summed per item, grouped by item_name, showing only items whose summed total exceeds 5000.00, sorted from highest total down to lowest.

Part B — MongoDB (Module 4) [3 Marks]**Collection Design & Insertion:**

Design an equivalent MongoDB collection named food_orders with fields tableNumber, itemName, and amount. Write an insertOne() call for the same order used above.

Query / Aggregation:

Write an aggregation pipeline on food_orders that groups documents by itemName, sums the amount field using \$sum, filters with \$match to keep totals exceeding 5000, and sorts the results in descending order of total.

QUESTION 4: SPRING BOOT REST ARCHITECTURE [20 Marks]

Build a mini Spring Boot stack that validates kitchen stock levels before a new order is accepted.

A. Entity Layer [5 Marks]

Write a FoodOrder class marked with standard JPA annotations mapping to a table named food_orders, with fields corresponding to orderId (primary key), tableNumber, itemName, and amount.

B. Repository Layer [5 Marks]

Write a FoodOrderRepository interface extending JpaRepository for the FoodOrder entity.

C. Service Layer [5 Marks]

Write an OrderValidationService class containing a method with the exact signature below. It receives a fixed array of 8 current stock levels for the ingredients an order requires.

```
public boolean validateStock(int[] itemStockLevels, int minimumStock) throws Exception
```

The method must loop through all 8 stock levels. If ANY level is strictly less than minimumStock, the order cannot be fulfilled and the method returns false immediately. If every ingredient has sufficient stock, it returns true once the loop completes.

D. Controller Layer [5 Marks]

Write an OrderController class mapped at /api/orders. Create a POST endpoint handler method named placeOrder that receives itemStockLevels and minimumStock via query string parameters.

The Confusion Trap:

If the service validation returns false (insufficient stock for the order), the controller must respond with an HTTP Status Code of 406 Not Acceptable. If validation passes, it must return 200 OK.

— END OF QUESTION PAPER —